

CAM-Guided Saliency Cutout and Image-Based Malware Classification

Yasaman Ebrahimi^{*†} Martin Jureček[†] Mark Stamp^{*§}

August 13, 2026

Abstract

Dropout regularization is commonly used to reduce overfitting by removing parts of a neural network during training. For Convolutional Neural Networks (CNN), cutouts serve a somewhat analogous purpose. Cutouts can be implemented as data augmentation: the original training image is retained, and additional copies are created with regions removed. In this chapter, we test whether cutout placement can be improved by using High-Resolution Class Activation Mapping (HiResCAM). We compare four controlled training conditions: no cutout, standard random cutout, low-saliency cutout, and high-saliency cutout. We experiment using grayscale malware images from the RawMal-TF dataset (17 families with 1,000 samples per family), and for comparison to natural images, we experiment with the well-known CIFAR-100 dataset. All experiments are based on ResNet18 with 100 training epochs. For the cutout experiments, we test cutout areas of 5%, 10%, 20%, and 30%, and we consider $M \in \{4, 8\}$ augmented copies per original training image. The RawMal-TF results are slightly worse for all three cutout cases (random, high and low saliency) as compared to no cutouts. In contrast, our CIFAR-100 experimental results improve slightly under low-saliency cutout. These results suggest that the value of saliency-guided cutout is domain dependent, and that malware images should not be treated as equivalent to natural images.

Keywords: HiResCAM · ResNet18 · RawMal-TF · CIFAR-100 · Cutout regularization · Malware classification · Convolutional Neural Networks

1 Introduction

Image-based malware classification is a process that turns binary files into images so that computer vision models can be used to analyze malware. Convolutional Neural

^{*}Department of Computer Science, San Jose State University

[†]Faculty of Information Technology, Czech Technical University in Prague

[‡]yasi.ebrahimi0@gmail.com

[§]mark.stamp@sjsu.edu

Network (CNN) models often achieve state-of-the-art results on challenging malware classification tasks. The main goal of this research is to evaluate whether cutout regularization guided by High-Resolution Class Activation Mapping (HiResCAM) improves image-based malware classification, and to compare this behavior with analogous results on natural-image datasets.

Standard cutout removes a randomly selected region from an image during training, which encourages a CNN to avoid relying too heavily on a single local region [6]. In this work, we keep the structure of standard cutout but change the rule used to choose the cutout location. Instead of selecting a square cutout region randomly, we use HiResCAM saliency maps from a trained teacher model to choose either a low-saliency region or a high-saliency region. The goal is to determine whether saliency can guide a cutout-style data augmentation method in a way that improves classification results.

The distinction between permanent masking and cutout augmentation is important. A permanent saliency mask tests whether the image can be simplified by deleting regions. In contrast, a cutout regularizer tests whether a model becomes more stronger when it is forced, during training, to classify partially occluded copies of the original images. These are different experimental questions. The present study therefore compares saliency-guided cutout directly against standard random cutout and a no-cutout baseline. This makes random cutout the primary control condition, since it uses the same number of augmented copies and the same cutout area, but does not use saliency to select the square cutout region.

Our main malware dataset is a collection of malware images derived from the RawMal-TF dataset [2, 3]. RawMal-TF is ideal for this study because it contains a large collection of malware samples labeled by family, and because each sample can be represented through multiple image transformations. Based on the goal of a controlled experiment, this chapter limits the RawMal-TF study to one image type, namely, standard grayscale images. For comparison, we conduct analogous experiments with the CIFAR-100 [15] natural-image dataset.

The research questions addressed in this chapter are the following:

- Does saliency-guided cutout improve malware family classification based on grayscale images, as compared with no cutout?
- Does saliency-guided cutout improve over standard random cutout when the number of cutout copies and cutout area are held constant?
- Is low-saliency cutout or high-saliency cutout more effective for malware image classification?
- Are the conclusions robust across multiple seed values and cutout areas?
- Is the behavior observed on malware images consistent with the behavior observed on natural images?

The main finding is negative for the malware task. That is, for the grayscale RawMal-TF images, the no-cutout ResNet18 baseline attains the best mean validation accuracy among all tested conditions. Standard random cutout, low-saliency cutout, and high-saliency cutout all reduce mean validation accuracy on this malware dataset. The expanded seed and cutout-area sweeps show that low-saliency cutout is sometimes slightly better than random cutout and sometimes slightly

worse, while high-saliency cutout is generally worse than random cutout. This result is interesting, because it shows that cutouts do not simply transfer from generic image regularization to malware images. The analogous CIFAR-100 results provide a useful contrast: low-saliency cutout improves over no cutout for several settings, while high-saliency cutout is consistently poor. This contrast suggests that saliency-guided cutout is potentially useful, but its usefulness depends on the image domain. These results also highlight that malware images are distinct from natural images, and hence techniques developed for natural images may not carry over to the field of malware analysis.

The remainder of this chapter is organized as follows. Section 2 covers relevant background topics, including related work. In Section 3 we outline the methodology used in subsequent experiments. Our experimental results are given in Section 4. Section 5 discusses the main findings, limitations, and implications for our malware-image experiments, along with suggestions for future work. In Section 6, we conclude the chapter with a summary of our main results.

2 Background and Related Work

This section reviews the main ideas needed to interpret our experiments. We first discuss dropout, cutout, and saliency-guided masking, as related to regularization and masking strategies. We then describe image-based malware classification and discuss why CAM-guided masking might be expected to behave differently for malware images than for natural images.

2.1 Dropout, Cutout, and Data Augmentation

Dropout regularization is a common method used to reduce overfitting during neural network training. It works by randomly removing hidden and input units during training, which discourages the model from depending too much on specific features and can improve generalization [23]. Cutout applies a somewhat similar idea to image data by masking random spatial regions of the input image during training [6]. For convolutional neural networks, this can help the model learn from different visual patterns instead of focusing too much on one region.

The standard cutout setting is most naturally viewed as data augmentation. A training image is retained in its original form, and additional training copies are created, typically with one square region removed. The model is then trained on both the original and occluded images. Note that validation and test images are not occluded, as the purpose is to improve a model that will later be evaluated on the original distribution. Therefore, a cutout method is successful only if it improves validation accuracy on the original images, not merely if the model learns to classify artificially masked images.

The experiments in this chapter use a simple and controlled version of this idea. When cutout is enabled, each original training sample contributes the original image plus M augmented cutout copies. The values $M = 4$ and $M = 8$ are tested. For

all cutout conditions, cutout areas of 5%, 10%, 20%, and 30% of the image area are evaluated. The random, low-saliency, and high-saliency settings differ in how the cutout region is selected, which is the core comparison needed for our research problem.

2.2 CAM and HiResCAM

Instead of masking random image areas, we propose a Class Activation Map (CAM) based method that can be used to identify important regions. Grad-CAM produces a class-discriminative heatmap by using gradients from a target convolutional layer to identify image regions that contribute to a prediction [22]. HiResCAM follows the same general approach, but it is designed to provide more faithful explanations by avoiding the gradient-averaging step used in Grad-CAM [8]. For a selected class, HiResCAM records the activations from a target convolutional layer and computes the gradient of the class score with respect to those activations. The activation at each spatial location is multiplied by its corresponding gradient, and these activation-gradient products are summed across the feature-map channels. The resulting heatmap is then passed through a ReLU operation, normalized, and resized to match the input image size. Larger heatmap values indicate regions that contribute more strongly to the model’s prediction, while smaller values indicate regions that contribute less strongly.

A recent malware-visualization study used CAM methods, including HiResCAM, to interpret malware image classifiers and to guide masking strategies [5]. This is part of a broader explainable-malware-analysis literature in which saliency maps and post hoc explanations are used to inspect classifier behavior [10, 18, 20]. Other saliency and perturbation-based approaches, such as RISE and deletion/insertion tests, further show that the choice of explanation method can affect how image evidence is interpreted [11, 21]. That work motivates the idea that CAM maps may contain useful information about which image regions matter for malware classification. However, the present work uses CAM maps for a different purpose. We are not testing whether removing low-saliency pixels produces a cleaner image; instead, we are testing whether CAM maps can guide cutout augmentation in a way that is analogous to standard random cutout.

A crucial distinction in this study is between low-saliency cutout and high-saliency cutout. Low-saliency cutout removes a square region selected from areas that the teacher model treats as less important. This tests whether the model benefits from occlusions placed in regions that are expected to be less informative. High-saliency cutout removes a square region selected from areas that the teacher model relies on most strongly. This is closer to the regularization intuition behind cutout: hiding highly informative regions may force the student model to learn from other available features. Although both strategies come from the same HiResCAM heatmap, they represent different research questions and should not be construed as the same type of masking.

2.3 Image-Based Malware Classification

Malware image classification treats executable bytes or extracted binary features as image-like structures that can be used by computer vision models. Early work on malware visualization showed that malware binaries can be converted into grayscale images and classified using visual patterns [19]. Subsequent work has considered handcrafted image descriptors, transfer learning, convolutional neural networks, and other deep learning approaches for image-based malware classification [4, 14, 24, 26]. More recent work has shown that different binary-to-image transformations can produce different visual representations of the same malware sample. Note that there is no universally accepted “best” image transformation for malware classification [1, 2].

The main malware dataset used in this study is RawMal-TF, which contains malware samples labeled by type and family [3]. The malware images used in our experiments consist of 17 malware families from RawMal-TF, with 1,000 malware samples per family. For each malware sample, eight distinct image representations are available, giving 17,000 malware samples and 136,000 total images [2]. In the present experiments, we do not consider all eight image representations. Instead, we select only the grayscale image representation and use this as our malware-image setting.

This setting is important because saliency in a malware image does not necessarily have the same meaning as saliency in a natural image. In natural images, highly activated regions may correspond to human-interpretable objects or visual features, while in malware images, such regions may instead reflect byte layout, section structure, padding, or various artifacts created during the image transformation process. Because of this difference, a CAM-guided masking strategy that works well for medical images or natural-image classification may not directly transfer to malware image classification. Therefore, we treat saliency-guided cutout as a method that must be tested within the malware-image setting rather than assumed to work in the same way across different image domains.

2.4 Model Scope

All reported experiments are based on ResNet18 [12]. This is a deliberate scope restriction in the current controlled study. ResNet18 is a suitable first architecture because it is a standard CNN baseline, trains efficiently, and is widely used in image-classification experiments. The model registry adapts the classifier head to the number of classes in each dataset. For small images, the implementation also uses a small-image stem rather than the larger ImageNet-style stem. Since the current results are ResNet18-only, this chapter does not claim that the conclusions are architecture independent. Instead, the results should be interpreted as a controlled ResNet18 study that establishes whether saliency-guided cutout is sufficiently promising to justify broader model sweeps.

3 Methodology

In this section, we provide relevant details on our experimental design. We also discuss the datasets that we use.

3.1 Experimental Conditions

In our experiments, we compare four training scenarios, as summarized in Table 1. The no-cutout scenario establishes the baseline for a dataset and the model. The random cutout scenario is the standard cutout regularization control. The low-saliency and high-saliency scenarios are our proposed CAM-guided variants.

Table 1: Training conditions for cutout experiments

Condition	Training data	Cutout region selection
No cutout	Original training images	No region is removed
Random cutout	Original images plus M cutout copies	Square region selected randomly
Low-saliency cutout	Original images plus M cutout copies	Square region selected from low-saliency candidate windows using teacher HiResCAM map
High-saliency cutout	Original images plus M cutout copies	Square region selected from high-saliency candidate windows using teacher HiResCAM map

Recall from Section 2.1 that, for each dataset, the cutout conditions are evaluated with $M = 4$ and $M = 8$. For both values of M , we experiment with square cutout regions that cover approximately 5%, 10%, 20%, or 30% of the input image. The augmented training set therefore contains the original image plus four or eight occluded copies, depending on the value of M . Validation data is never cut out, which is essential for interpreting the result as regularization rather than as matched-distribution training on a transformed validation set.

3.2 Pipeline Overview

Figure 1 illustrates our experimental pipeline. First, a no-cutout ResNet18 model is trained on the original training images. This model serves two roles: it is the no-cutout baseline, and it provides the teacher checkpoint used to compute HiResCAM maps for CAM-guided cutout. For the random condition, no teacher is needed. For the low-saliency and high-saliency conditions, the teacher model generates saliency maps for training images. Candidate square windows are scored by saliency, and the cutout location is chosen from the low-scoring or high-scoring candidates depending on the condition. The student model is then trained on the original images plus the selected cutout copies. Finally, each trained student model is evaluated on the original validation split.

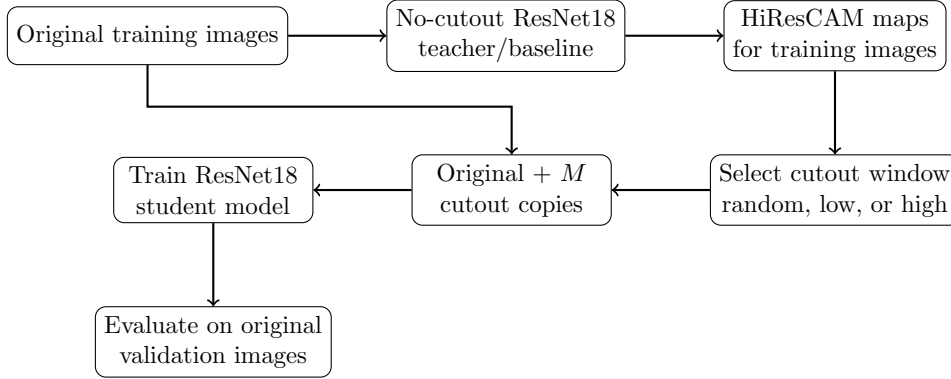


Figure 1: Controlled cutout pipeline

3.3 CAM-Guided Cutout Implementation

The saliency-guided cutout implementation is designed to mimic standard cutout as closely as possible. For a given image, cutout size, and augmentation index, the method selects one square window and fills it with normalized (i.e., black) value. The random condition selects this window randomly, while the CAM-guided conditions select it using a HiResCAM saliency map.

For CAM-guided cutout, the teacher model is the no-cutout ResNet18 checkpoint trained with the same dataset and seed. The implementation automatically resolves the final convolutional target layer and computes a HiResCAM map for the teacher’s predicted class. The heatmap is normalized and resized to the input image size. Candidate cutout windows are then evaluated according to the average saliency inside the square. In the low-saliency condition, the method forms a candidate set from the lowest-scoring 10% of valid windows. In the high-saliency condition, the candidate set is formed from the highest-scoring 10%. One window is selected from the appropriate candidate set using a specified, copy-specific seed value. Selecting from a candidate set, rather than always selecting the absolute minimum or maximum window, preserves diversity among the M augmented copies.

Because CAM generation is expensive, the implementation includes saliency and window caching. CAM saliency maps are cached as CPU tensors under a CAM cache directory, and selected window coordinates are cached under a window subdirectory. This cache is especially important for 224×224 malware-image datasets, where many candidate windows must be evaluated. The augmented copies are static: a sample index and augmentation index map to the same selected window in every epoch. Caching prevents repeated CAM pooling and candidate-window selection across epochs. This makes the low-saliency and high-saliency runs feasible while preserving the intended experimental design.

3.4 Repository and Code Organization

Our code can be found in the public CAMRegularization repository [9]. The active workflow is standard training through `train.py`, with dataset and model selection

handled by registries. The main source files implement training, evaluation, cutout augmentation, CAM generation, dataset loading, model construction, logging, utility functions, plotting, and CAM-cutout validation.

The `train.py` script parses the run configuration, loads the dataset, builds the selected model, optionally loads the teacher checkpoint for CAM-guided cutout, wraps the training dataset with the cutout augmentation dataset, trains for the requested number of epochs, and writes the run outputs. The `engine.py` file contains the epoch-level training and evaluation routines. The `cutout.py` file implements the augmented dataset wrapper for random, low-saliency, and high-saliency cutout. The `cam_masking.py` file implements HiResCAM saliency generation. The `dataset_registry.py` and `model_registry.py` files isolate dataset loading and model construction so that the same training pipeline can be used for RawMal-TF and CIFAR-100.

Each committed run folder contains three artifacts: `config.json`, `metrics.csv`, and `metrics_plot.png`. The configuration file records the resolved inputs for the run. The metrics file records per-epoch training loss, training top-1 accuracy, validation loss, validation top-1 accuracy, learning rate, and evaluation split. The metrics plot visualizes training and validation loss and accuracy across epochs. The results reported in this chapter are computed from these per-run metrics files and from the repository summary files generated from them.

The current archive contains 168 physical run folders. No-cutout runs are repeated under area directories for matched folder organization even though cutout area does not apply to the baseline. The summary procedure compares hashes, selects one baseline observation per dataset and seed, and excludes duplicate copies. After this deduplication, 150 runs are used for analysis: 144 augmented runs and six no-cutout baselines. All expected augmented combinations are present. One differing duplicate CIFAR-100 baseline copy is excluded, while the repeated modal baseline is retained. This handling prevents baseline duplication from inflating seed counts or paired comparisons.

3.5 Datasets

Our experiments use the two datasets summarized in Table 2. RawMal-TF is the primary dataset and provides the malware-family classification setting used to evaluate low- and high-saliency cutout [2, 3]. CIFAR-100 is included as a natural-image comparison dataset so that the cutout behavior can be compared with a standard computer-vision benchmark [15].

For RawMal-TF, the dataset is loaded through the `drive_zip` registry entry. The run configuration sets `grayscale=true` and uses the include pattern `_grayscale.tiff$`. This ensures that the RawMal-TF experiment is limited to grayscale images and does not mix the eight available image representations.

Table 2: Datasets analyzed

Dataset	Type	Characteristics	Role
RawMal-TF	Malware images	17 malware families	Main empirical focus
		1,000 samples per family grayscale images	
CIFAR-100	Natural images	100 classes standard benchmark	Natural-image comparison

3.6 Training Configuration and Metrics

Across all reported experiments, we use ResNet18, seed values 42, 43, and 44, 100 epochs, batch size 128, SGD optimization, learning rate 0.1, momentum 0.9, weight decay 0.0005, a cosine learning-rate scheduler, five warmup epochs, automatic mixed precision, and deterministic training transforms. Each run uses a validation split value of 0.1. Validation data remain unmasked. No held-out test results are reported in this chapter because the committed `metrics.csv` files contain validation metrics only. For CAM-guided cutout, the teacher model is ResNet18 and the teacher checkpoint is the no-cutout run for the same dataset and seed. The CAM layer is selected automatically.

The main reported metric is best validation top-1 accuracy. We report the mean and sample standard deviation across the three seeds for each value of M , cutout area, and condition combination. We also compute paired differences within each seed before averaging, so that low-saliency and high-saliency cutout are compared with the corresponding random cutout run under the same dataset, seed, area, and M .

Several secondary metrics characterize convergence and stability. Final validation accuracy is the value at epoch 100. Best-to-final degradation measures the decline from the best epoch to epoch 100. Normalized validation area under the learning curve (AULC) summarizes performance over all 100 epochs. The final-20 standard deviation measures late-training variation, maximum validation drawdown records the largest decline from a previously achieved running best, and collapse-event count records entries into a state at least five percentage points below the previous running best. These secondary metrics help distinguish a condition that produces one favorable epoch from a condition that performs consistently across training. With only three seeds, confidence intervals and variance estimates are exploratory, and we avoid strong statistical-significance claims.

4 Results

In this section, we first present our main experimental results involving the RawMal-TF dataset. Then we give results for analogous experiments on CIFAR-100. We conclude this section with a cross-dataset summary and representative learning curves.

4.1 RawMal-TF Malware Results

RawMal-TF experiments are the main focus of this study. Across the expanded seed and area sweep, saliency-guided cutout did not improve grayscale malware image classification. The no-cutout baseline obtained the highest mean best validation accuracy, and every cutout condition produced a lower mean accuracy.

The no-cutout baseline reaches $72.83\% \pm 0.16\%$ mean best validation accuracy. The best cutout result is random cutout with $M = 4$ and 30% area, which reaches $71.55\% \pm 0.45\%$. The strongest low-saliency result is $71.43\% \pm 1.12\%$ for $M = 4$ and 30% area. Thus, the best random cutout condition is 1.28 percentage points below the no-cutout baseline, and the best low-saliency condition is 1.39 percentage points below the baseline.

Table 3 gives all RawMal-TF area and M combinations. The final two columns give the mean paired change relative to the matched random cutout control. Positive values favor the CAM-guided condition.

Table 3: RawMal-TF mean best validation accuracy across seeds 42, 43, and 44 (accuracy entries are mean $\pm$ sample standard deviation in percent; paired changes are percentage points; the no-cutout baseline is 72.83 ± 0.16)

M	Area (%)	Random	Low saliency	High saliency	Low-random	High-random
4	5	70.87 ± 0.48	70.94 ± 0.63	70.66 ± 1.03	+0.08	-0.21
4	10	70.88 ± 0.66	71.02 ± 0.50	70.42 ± 0.62	+0.14	-0.46
4	20	70.62 ± 0.90	71.08 ± 0.32	69.68 ± 0.74	+0.46	-0.95
4	30	71.55 ± 0.45	71.43 ± 1.12	69.54 ± 0.71	-0.12	-2.00
8	5	70.59 ± 0.43	70.02 ± 0.57	70.01 ± 0.24	-0.56	-0.58
8	10	70.97 ± 0.81	70.63 ± 0.57	69.84 ± 0.77	-0.34	-1.13
8	20	70.93 ± 0.33	70.77 ± 1.09	69.75 ± 0.60	-0.16	-1.18
8	30	71.23 ± 0.62	70.83 ± 0.85	69.67 ± 0.30	-0.40	-1.56

The low-saliency comparison is small and seed-sensitive. Across the eight matched area/ M cells, low-saliency minus random ranges from -0.56 to $+0.46$ percentage points. One cell is positive for all three seeds, one cell is negative for all three seeds, and the remaining six cells have mixed signs. Therefore, these experiments do not support a reliable low-saliency advantage over random cutout on RawMal-TF.

High-saliency cutout is even less favorable. Its mean paired effect relative to random cutout ranges from -2.00 to -0.21 percentage points. Six of the eight matched cells are negative for all three seeds, and the other two are mixed. The largest deficit occurs for $M = 4$ and 30% area. This is consistent with the possibility that masking the region most strongly associated with the teacher prediction removes useful malware-family evidence without providing a compensating regularization benefit.

Figure 2 plots validation error rather than validation accuracy, so that differences among conditions are easier to observe. The no-cutout error is lower than every cutout curve. Low-saliency and random cutout remain close to one another, while high-saliency error generally increases as the cutout area grows.

Figure 3 emphasizes the primary control comparison. The low-saliency curve remains near zero, which reflects the inconsistent and small differences in Table 3.

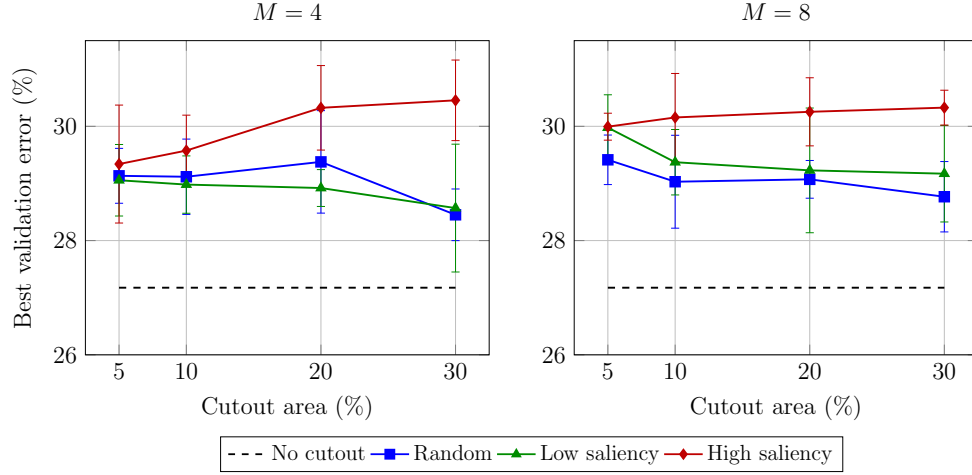


Figure 2: RawMal-TF best validation error by cutout area (lower is better; error bars denote sample standard deviation across seeds)

The high-saliency curve is mostly below zero and becomes particularly unfavorable for large masks.

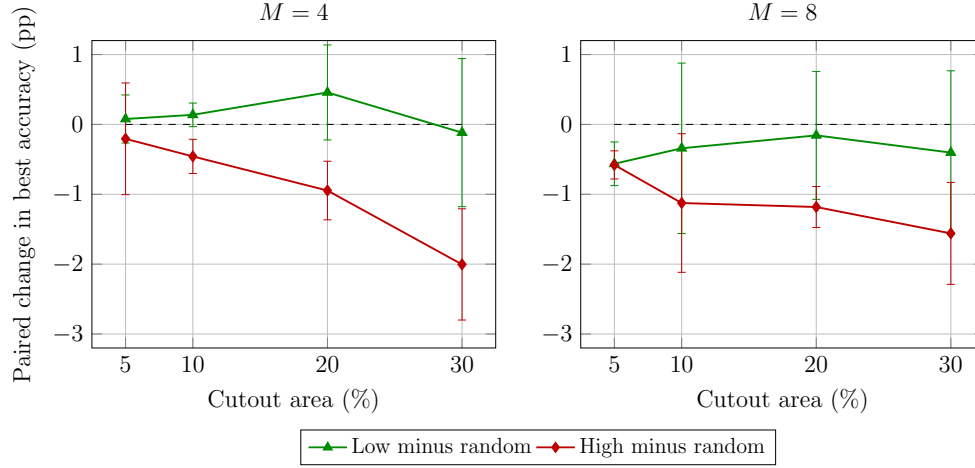


Figure 3: RawMal-TF paired effects relative to matched random cutout (positive values favor CAM-guided cutout; error bars denote sample standard deviation across seeds)

Figure 4 gives a compact view of the same matched comparisons across both augmentation multiplicities and all four cutout areas. Each cell is the three-seed mean change in best validation accuracy relative to the random-cutout run with the same seed, area, and value of M . Values near zero dominate the low-saliency columns, whereas the high-saliency columns are negative throughout and become more unfavorable for larger masks.

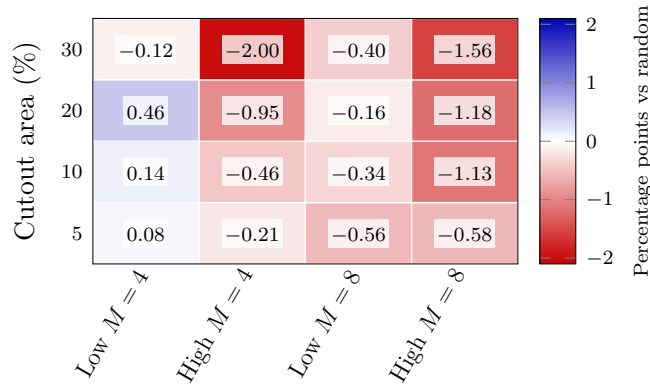


Figure 4: RawMal-TF paired-effect heatmap (blue cells favor saliency-guided cutout, red cells favor matched random cutout, and near-white cells indicate little difference)

4.2 CIFAR-100 Results

CIFAR-100 is included as a natural-image comparison dataset. For this dataset, our no-cutout baseline reaches $62.65\% \pm 0.57\%$ for the mean best validation accuracy. Low-saliency cutout with $M = 4$ and 10% area is the best CIFAR-100 result at $63.51\% \pm 0.36\%$, which is 0.86 percentage points above the no-cutout baseline and 1.25 percentage points above the corresponding random cutout control. Low-saliency cutout with $M = 8$ also performs well at 5% and 10% area. These CIFAR-100 results are useful because they show that saliency-guided cutout can have a positive effect in at least one natural image setting, which is in contrast to our malware experiments, above.

Table 4 gives the complete CIFAR-100 sweep. Low-saliency cutout is especially useful when the mask is large enough that random placement becomes destructive. At 20% and 30% area, low-saliency cutout substantially outperforms matched random cutout for both values of M . However, low-saliency cutout at 30% area remains below the no-cutout baseline, which shows that “safer” placement does not completely eliminate the damage caused by very large occlusions.

Table 4: CIFAR-100 mean best validation accuracy across seeds 42, 43, and 44 (accuracy entries are mean $\pm$ sample standard deviation in percent; paired changes are percentage points; the no-cutout baseline is 62.65 ± 0.57)

M	Area (%)	Random	Low saliency	High saliency	Low-random	High-random
4	5	62.76 ± 0.46	62.88 ± 0.35	60.80 ± 0.50	+0.12	-1.95
4	10	62.26 ± 0.34	63.51 ± 0.36	58.85 ± 0.52	+1.25	-3.41
4	20	59.47 ± 0.79	63.05 ± 0.35	54.07 ± 0.49	+3.58	-5.39
4	30	53.98 ± 0.71	59.95 ± 0.73	47.26 ± 0.29	+5.97	-6.72
8	5	62.37 ± 0.38	63.22 ± 0.10	60.85 ± 0.37	+0.85	-1.52
8	10	62.27 ± 0.16	63.31 ± 0.77	58.15 ± 0.14	+1.04	-4.11
8	20	59.07 ± 0.50	62.75 ± 0.39	51.99 ± 0.60	+3.68	-7.08
8	30	51.68 ± 0.86	59.59 ± 0.55	45.62 ± 0.27	+7.91	-6.06

The high-saliency CIFAR-100 result is the opposite of the low-saliency result. High-saliency cutout is worse than matched random cutout in every area/ M cell and for every seed. The paired deficit ranges from 1.52 to 7.08 percentage points. This is consistent with the intuition that hiding the most important object regions can be counterproductive when dealing with natural images. The low-saliency result, by contrast, suggests that saliency can identify regions that can be occluded with less damage to the main object signal.

Figure 5 uses validation error to make these differences visible. Random cutout and high-saliency cutout deteriorate rapidly as area increases. Low-saliency cutout is much more robust to area, and its 10% settings produce the lowest mean error.

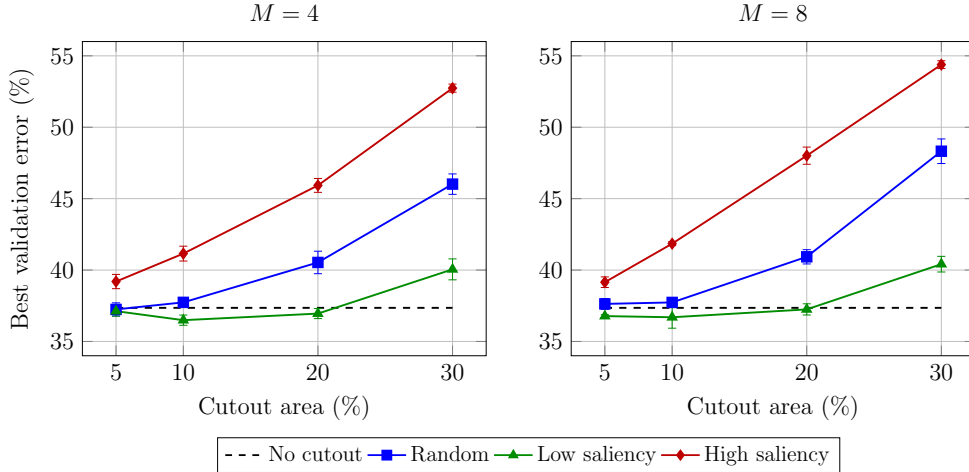


Figure 5: CIFAR-100 best validation error by cutout area (lower is better; error bars denote sample standard deviation across seeds)

Figure 6 presents the same result relative to matched random cutout. The low-saliency advantage grows as area increases, while high-saliency cutout remains consistently unfavorable. Seven of the eight low-saliency area/ M cells are positive for all three seeds; the 5%, $M = 4$ cell has mixed signs. In contrast, all eight high-saliency cells are negative for all three seeds.

Figure 7 summarizes the CIFAR-100 paired effects in the same format. The low-saliency columns become increasingly positive as the cutout area grows, while every high-saliency cell is negative. This heatmap makes the interaction between placement rule, mask area, and augmentation multiplicity visible at a glance.

Figure 8 exposes the individual seed-level low-saliency effects that underlie the means and error bars. The CIFAR-100 effect is positive and increasingly large for most settings, whereas the RawMal-TF traces remain close to zero and change sign across seeds. This view makes the contrast between a robust natural-image trend and a seed-sensitive malware-image result explicit.

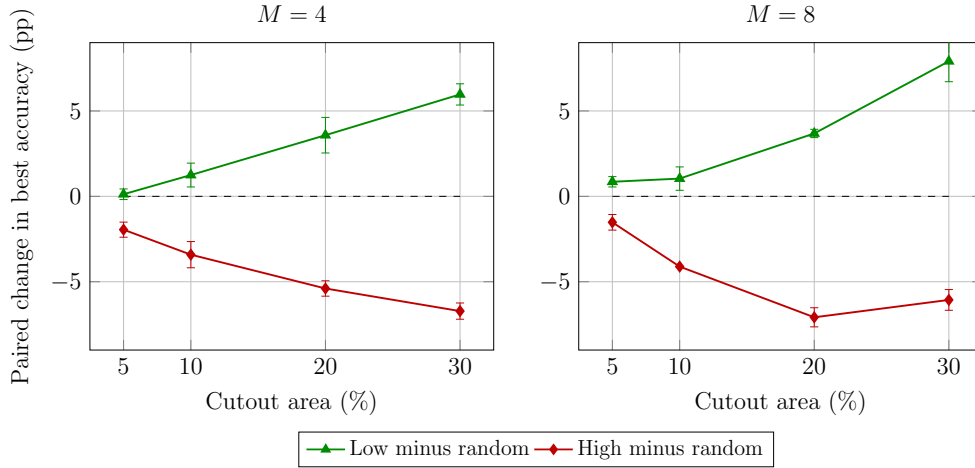


Figure 6: CIFAR-100 paired effects relative to matched random cutout (positive values favor CAM-guided cutout; error bars denote sample standard deviation across seeds)

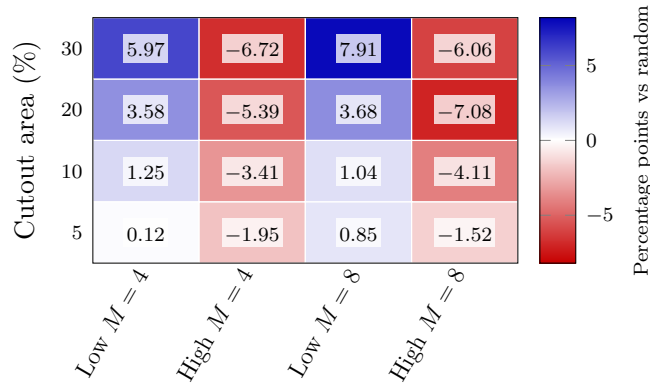


Figure 7: CIFAR-100 paired-effect heatmap (blue cells favor saliency-guided cutout, red cells favor matched random cutout, and near-white cells indicate little difference)

4.3 Cross-Dataset Summary

Table 5 summarizes the main across-seed findings. The overall pattern is not consistent across image types: RawMal-TF is best with no cutout, while CIFAR-100 is best with low-saliency cutout at $M = 4$ and 10% area. The most important difference is not simply the absolute accuracy level, but the relationship between CAM-guided placement and the matched random control.

Figure 9 visualizes the same cross-dataset comparison and makes the different domain-level rankings immediately visible. Low-saliency cutout supplies the best CIFAR-100 mean, whereas the RawMal-TF no-cutout baseline remains clearly above the best setting from every cutout family.

The secondary metrics lead to the same broad conclusion. For RawMal-TF, the

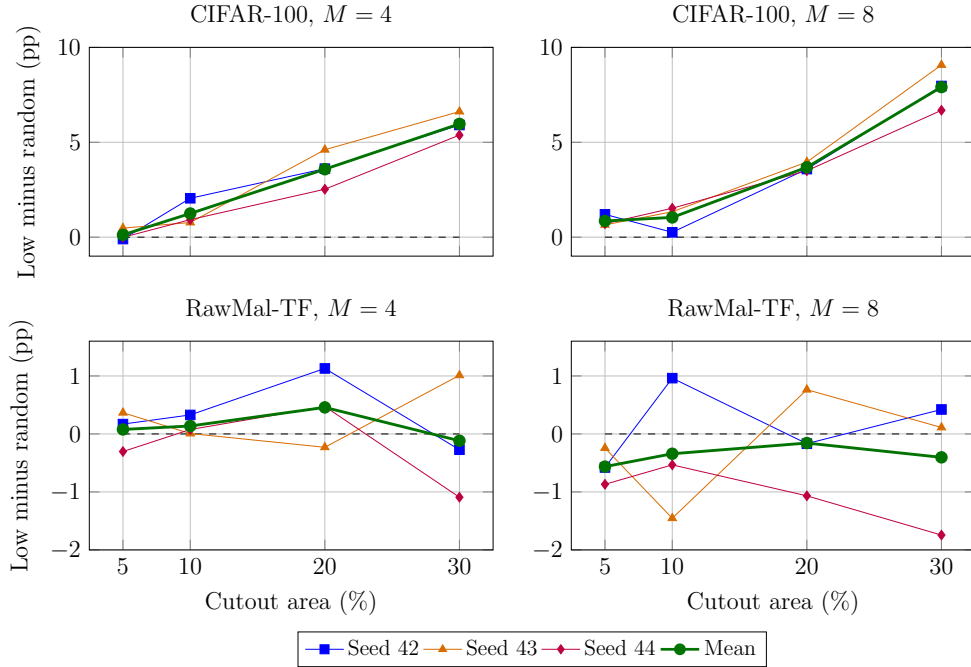


Figure 8: Seed-level paired effects for low-saliency cutout relative to matched random cutout (the thick green curve is the three-seed mean)

Table 5: Cross-dataset summary of best mean validation results

Dataset	No cutout	Best cutout results		
		Random	Low saliency	High saliency
CIFAR-100	62.65 ± 0.57	62.76 ± 0.46	63.51 ± 0.36	60.85 ± 0.37
RawMal-TF	72.83 ± 0.16	71.55 ± 0.45	71.43 ± 1.12	70.66 ± 1.03

no-cutout baseline has a mean normalized validation AULC of 68.05%, compared with 67.47% for the best random condition and 67.44% for the best low-saliency condition. For CIFAR-100, low-saliency cutout can improve peak accuracy without producing an equally large AULC improvement: the best peak condition, low saliency with $M = 4$ and 10% area, has a mean AULC of 53.05%, the same as for the no-cutout baseline. Low saliency with $M = 8$ and 5% area has the highest selected AULC at 53.20%. Thus, the CIFAR-100 advantage is genuine in peak validation accuracy, but the learning-curve summary indicates that the advantage is concentrated in the later portion of training. Figure 10 places these AULC values beside best-to-final degradation for the same selected conditions. The figure reinforces that peak validation accuracy and whole-curve behavior are related but distinct: the best CIFAR-100 low-saliency setting improves the peak while remaining close to the baseline in AULC, and none of the selected RawMal-TF cutout settings exceeds the baseline AULC.

Figure 11 presents representative individual run-folder curves using validation error. The curves are not used in place of the three-seed aggregates; they illustrate

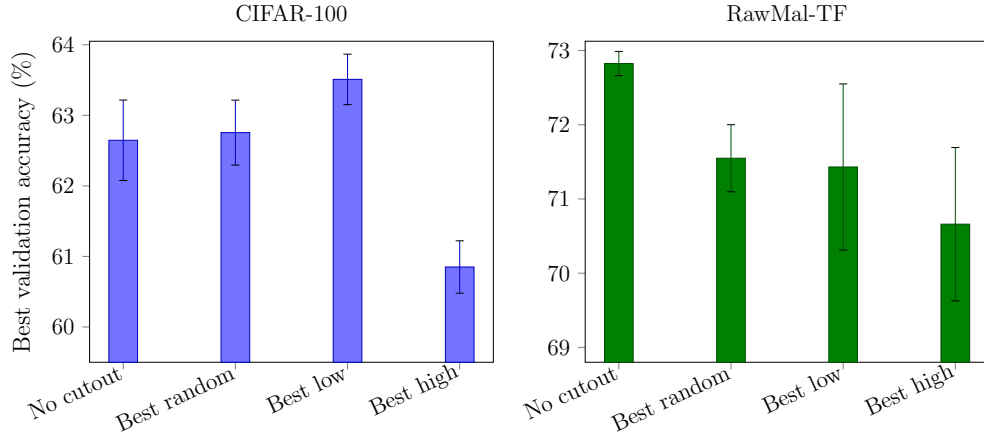


Figure 9: Best mean validation accuracy within each method family. Each cutout bar represents that method’s best area and M setting for the corresponding dataset.

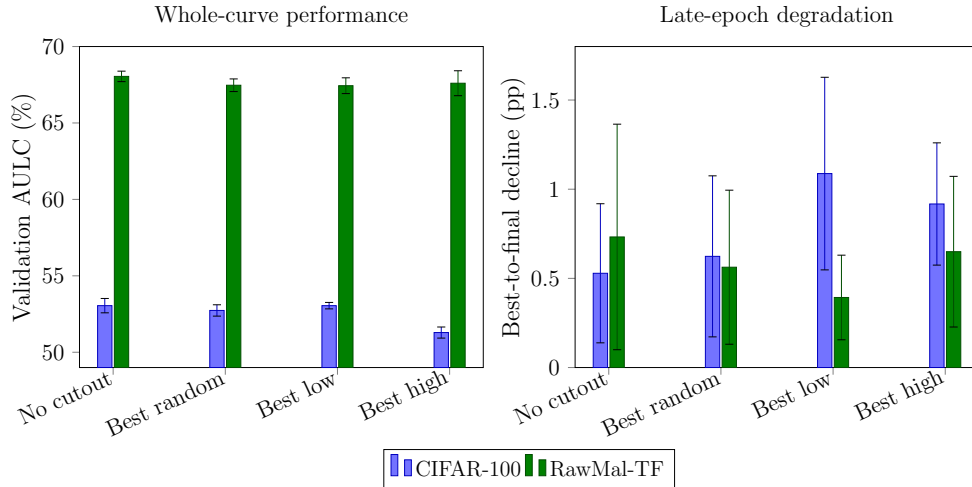


Figure 10: Secondary learning-curve metrics for the selected method-family representatives. Error bars denote sample standard deviation across seeds.

typical late-training behavior. The CIFAR-100 low-saliency condition can reach a lower validation error than the no-cutout run, while the strongest RawMal-TF random condition remains above the RawMal-TF no-cutout curve.

5 Discussion

In this section we expand on the implications of our experimental results. We also consider limitations of the research presented in this chapter.

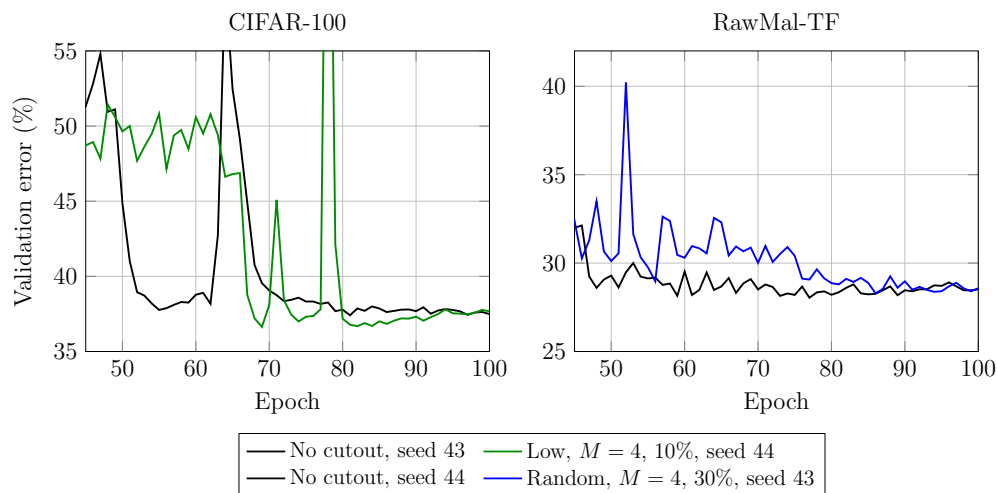


Figure 11: Representative individual-run validation error curves. Aggregate conclusions are based on all three seeds, not on these individual curves.

5.1 What the RawMal-TF Result Means

Our main RawMal-TF result is negative with respect to the original goal of improving malware classification with CAM-guided regularization. Neither low-saliency nor high-saliency cutout improves over the no-cutout baseline. More importantly, low-saliency cutout does not consistently improve over the standard random cutout control, and high-saliency cutout is generally worse. This is the most relevant comparison because random cutout is the direct control for cutout-style augmentation.

The same basic pattern is observed across seed values and across cutout areas ranging from 5% through 30%. The exact ranking of random and low-saliency cutout changes across some matched cells, but neither produces a mean result that reaches the no-cutout baseline. The strongest RawMal-TF cutout setting uses random placement, not saliency-guided placement.

One possible explanation is that local occlusion is less appropriate for grayscale malware images than for natural images. In a natural image, a local square may hide a patch of background, texture, or part of an object. In a malware image, a local square corresponds to a contiguous region of the transformed binary representation. Depending on the image-conversion method, such a region may encode structure that is important for family classification. Removing such structure may not encourage robust visual reasoning—it may simply damage a useful representation.

Another possible explanation is that saliency maps may not identify regions that are useful for cutout in the malware setting. A high-saliency region in a malware image may correspond to byte layout, packed content, padding, or other low-level artifacts, and a low-saliency region may still contain structural information needed by the classifier. Conversely, a low-saliency square may overlap zero padding or an already uniform region and therefore create only a weak augmentation. The metrics considered in this chapter do not account for zero-padding overlap or provide

effective perturbation measurements, so these possibilities remain hypotheses for future research.

5.2 Low-Saliency vs High-Saliency Cutout

Low-saliency and high-saliency cutout answer different questions. Low-saliency cutout asks whether the model benefits from occluding regions that the teacher considers less important. High-saliency cutout asks whether the model benefits from being forced to “look away” from the most important regions. Our results suggest that high-saliency cutout is the riskier strategy, as it is consistently worse than low-saliency cutout on CIFAR-100 and generally worse on RawMal-TF.

The CIFAR-100 result is especially informative. Low-saliency cutout at 10% area improves over no cutout, while high-saliency cutout sharply reduces performance. Low-saliency placement also protects performance when random masks become too large. This supports the idea that low-saliency regions can provide safer occlusion locations. However, the RawMal-TF result shows that this intuition does not automatically transfer to malware images. In the malware setting, even the best low-saliency condition is worse than no cutout, and its comparison with random cutout is seed-sensitive.

High-saliency cutout may also create a form of label inconsistency. The augmented image retains the original class label, but a large square intentionally removes the evidence the teacher most strongly associates with its prediction. On a natural image this can remove the object itself; on a malware image it can remove a discriminative byte region. Increasing the number or area of such masks therefore need not behave like beneficial regularization.

5.3 Effect of Cutout Area and Number of Copies

The area sweep reveals a strong domain difference. On CIFAR-100, random and high-saliency cutout deteriorate rapidly as area increases, while low-saliency placement is substantially more robust. At 20% and 30% area, the low-saliency advantage over random cutout becomes relatively large because saliency guidance avoids the most destructive locations. Nevertheless, 30% low-saliency masks still underperform the no-cutout baseline.

On RawMal-TF, changing the area does not produce a monotonic effect. The best random and low-saliency means occur at 30% area with $M = 4$, but both remain below no cutout. The low-minus-random effects remain small relative to their across-seed variability. Therefore, the area sweep does not reveal a RawMal-TF setting in which saliency guidance becomes reliably beneficial.

The comparison between $M = 4$ and $M = 8$ is descriptive rather than compute-matched. Each original image yields five training examples when $M = 4$ and nine when $M = 8$, so the number of augmented examples and optimizer updates changes with M . On RawMal-TF, the strongest cutout settings use $M = 4$. On CIFAR-100, both values can be useful for low-saliency placement, but $M = 8$ does not produce

a universally better result. Overall, this indicates that M should be treated as a hyperparameter rather than as a universally optimal setting.

5.4 Structure-Aware Masking for Malware Images

The negative RawMal-TF result suggests that a masking strategy designed for natural-image saliency may not exploit the structure of executable binaries. Malware images originate from byte sequences and executable formats, not from photographic scenes. A square selected only by image saliency ignores information such as file offsets, binary regions, entropy transitions, executable headers, and Portable Executable (PE) section boundaries.

A more appropriate malware-specific augmentation could align masks with known binary structure. For example, a structure-aware method could restrict or stratify masks by PE section, distinguish headers from code and data regions, avoid zero-padded areas, or mask contiguous byte ranges before image conversion. Another approach could combine saliency with structural constraints, such as selecting a low- or high-saliency region only within a specified PE section. These methods would preserve the controlled comparison with random masking while testing whether malware-specific information is more useful than purely image-based square saliency.

The present results should not be interpreted as showing that all intelligent masking is ineffective for malware. Instead, our results simply show that the current low- and high-saliency square strategies, transferred from natural-image reasoning, do not improve grayscale RawMal-TF classification.

5.5 Limitations and Future Work

The current experiments have several limitations. First, three seeds provide a substantially stronger basis than a single run, but the sample size remains small. Means, standard deviations, and t -based confidence intervals should be interpreted as exploratory. Additional seeds would be useful for small RawMal-TF paired effects.

Second, the current results are ResNet18-specific. This is useful for a controlled comparison, but it is conceivable that the same pattern may not hold for DenseNet, EfficientNet, ConvNeXt, vision transformers (ViT), Swin Transformers, or other architectures [7, 13, 16, 17, 25]. A broader study should repeat the same controlled comparison across multiple models.

Third, the current RawMal-TF experiment uses only grayscale images. Grayscale images are most common in malware analysis, and this restriction provides a reasonable first test case. However, future work should also consider different image types, while studying saliency within one representation at a time.

Fourth, the current archive supports validation accuracy, validation loss, and learning-curve stability summaries, but it does not contain held-out test accuracy, macro-F1, per-family metrics, confusion matrices, calibration, or sample-level predictions. These outputs should be added before making strong test-set or family-specific claims. The current chapter therefore reports only what is supported by the specific metrics considered.

Fifth, the same seed controls the data split and other sources of training randomness. Within-seed comparisons remain matched, but across-seed variability combines split sensitivity and optimization sensitivity. A future implementation should separate the split seed from model initialization, shuffling, and cutout placement.

Sixth, the current teacher generates HiResCAM for its predicted class using the final convolutional layer. Future work should compare predicted-class and ground-truth targets, HiResCAM and Grad-CAM, and earlier versus later target layers. Candidate-window percentages should also be varied.

Finally, the present $M = 4$ and $M = 8$ experiments are not compute-matched with no cutout or with each other. Future work should report wall-clock time, GPU time, and optimizer steps, and should include controls with an equal training budget. The present chapter should therefore be interpreted as a controlled augmentation sweep rather than as a training-efficiency study.

6 Conclusion

This study evaluated saliency-guided cutout regularization for image-based malware classification. The controlled experiments compared no cutout, standard random cutout, low-saliency cutout, and high-saliency cutout using ResNet18. For each cutout method, the training set contains the original images plus M augmented cutout copies, with $M = 4$ and $M = 8$ tested. The main malware experiment used grayscale RawMal-TF images and every condition was evaluated with seeds 42, 43, and 44 and cutout areas of 5%, 10%, 20%, and 30%, with 100 training epochs per run.

The RawMal-TF results show that saliency-guided cutout did not improve malware classification under this setup. The no-cutout baseline attained $72.83\% \pm 0.16\%$ mean best validation accuracy, while the best cutout condition, random cutout with $M = 4$ and 30% area, reached $71.55\% \pm 0.45\%$. Low-saliency cutout was sometimes slightly better and sometimes slightly worse than matched random cutout, and no low-saliency condition exceeded the no-cutout baseline. High-saliency cutout was generally harmful.

CIFAR-100 gave a different result. Low-saliency cutout with $M = 4$ and 10% area reached $63.51\% \pm 0.36\%$, compared with $62.65\% \pm 0.57\%$ for no cutout. Low-saliency placement also substantially outperformed random placement for large masks, while high-saliency cutout was consistently harmful. This contrast shows that the implementation can produce a positive saliency-guided result, but that the benefit does not transfer to the grayscale malware-image setting.

Overall, our results suggest that saliency-guided cutout is domain dependent. The method may be useful for natural images, but it does not currently improve grayscale RawMal-TF malware classification. This result is useful because it clarifies that malware image saliency should not be treated as equivalent to object saliency in natural images. For malware images, future masking strategies should incorporate executable structure, such as binary regions or PE-section boundaries, rather than

relying only on image saliency and square masks.

7 Data and Code Availability

The code and experimental results analyzed in this chapter are available in the public CAMRegularization GitHub repository at [9]. The repository for this chapter is commit `cd7f42fe9b3c3b52e291acf795e2dcc7e75bd14f`. The current run folders used for this research contain the resolved configuration, per-epoch metrics, and training plot for each ResNet18 run. Summary figures and tables are generated from the current `metrics.csv` files and repository summary CSV files.

References

- [1] Rishit Agrawal, Kunal Bhatnagar, Andrew Do, Ronnit Rana, Martin Jureček, and Mark Stamp. A comparison of selected image transformation techniques for malware classification. In Roberto Di Pietro, Karen Renaud, and Paolo Mori, editors, *Proceedings of the 12th International Conference on Information Systems Security and Privacy*, volume 2 of *ICISSP*, pages 334–344, 2026.
- [2] Rishit Agrawal, Kunal Bhatnagar, Andrew Do, Ronnit Rana, and Mark Stamp. A comparison of selected image transformation techniques for malware classification. <https://arxiv.org/abs/2509.10838>, 2025.
- [3] David Bálik, Martin Jureček, and Mark Stamp. RawMal-TF: Raw malware dataset labeled by type and family. <https://arxiv.org/abs/2506.23909>, 2025.
- [4] Niket Bhodia, Pratikkumar Prajapati, Fabio Di Troia, and Mark Stamp. Transfer learning for image-based malware classification. In Paolo Mori, Steven Furnell, and Olivier Camp, editors, *Proceedings of the 5th International Conference on Information Systems Security and Privacy*, *ICISSP*, pages 719–726, 2019.
- [5] Matteo Brosolo, Vinod P., and Mauro Conti. Through the static: Demystifying malware visualization via explainability. *Journal of Information Security and Applications*, 91:104063, 2025.
- [6] Terrance DeVries and Graham W. Taylor. Improved regularization of convolutional neural networks with cutout. <https://arxiv.org/abs/1708.04552>, 2017.
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, *ICLR*, 2021.
- [8] Rachel Lea Draelos and Lawrence Carin. Use HiResCAM instead of Grad-CAM for faithful explanations of convolutional neural networks. <https://arxiv.org/abs/2011.08891>, 2020.

- [9] Yasaman Ebrahimi. CAMRegularization, GitHub repository. <https://github.com/yasamanebrahimi-byte/CAMRegularization>, 2026.
- [10] Tristan Gomez and Harold Mouchère. Computing and evaluating saliency maps for image classification: a tutorial. *Journal of Electronic Imaging*, 32(2):020801, 2023.
- [11] Naofumi Hama, Masayoshi Mase, and Art B. Owen. Deletion and insertion tests in regression models. *Journal of Machine Learning Research*, 24:1–38, 2023.
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, CVPR, pages 770–778, 2016.
- [13] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, CVPR, pages 4700–4708, 2017.
- [14] Mugdha Jain, William B. Andreopoulos, and Mark Stamp. Convolutional neural networks and extreme learning machines for malware classification. *Journal of Computer Virology and Hacking Techniques*, 16(3):229–244, 2020.
- [15] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [16] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, ICCV, pages 10012–10022, 2021.
- [17] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, CVPR, pages 11976–11986, 2022.
- [18] Harikha Manthena, Shaghayegh Shajarian, Jeffrey C. Kimmell, Mahmoud Abdelsalam, Sajad Khorsandroo, and Maanank Gupta. Explainable artificial intelligence (XAI) for malware analysis: A survey of techniques, applications, and open challenges. *IEEE Access*, 13:61611–61640, 2025.
- [19] Lakshmanan Nataraj, Shankarapani Karthikeyan, George Jacob, and BS Manjunath. Malware images: Visualization and automatic classification. In *Proceedings of the 8th International Symposium on Visualization for Cyber Security*, pages 1–7, 2011.
- [20] Sadia Nazim, Muhammad Mansoor Alam, Syed Safdar Rizvi, Jawahir Che Mustapha, Syed Shujaa Hussain, and Mazliham Mohd Suud. Advancing malware imagery classification with explainable deep learning: A state-of-the-art approach using SHAP, LIME and Grad-CAM. *PLOS ONE*, 20(5):e0318542, 2025.

- [21] Vitali Petsiuk, Abir Das, and Kate Saenko. RISE: Randomized input sampling for explanation of black-box models. In *British Machine Vision Conference*, BMVC, page 151, 2018.
- [22] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *2017 IEEE International Conference on Computer Vision*, ICCV, pages 618–626, 2017.
- [23] Nitish Srivastava, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.
- [24] Mark Stamp. A selective survey of deep learning techniques and their application to malware analysis. In Mark Stamp, Mamoun Alazab, and Andrii Shalaginov, editors, *Malware Analysis Using Artificial Intelligence and Deep Learning*, pages 3–51. Springer, 2021.
- [25] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning*, ICML, pages 6105–6114, 2019.
- [26] Sravani Yajamanam, Vikash Raja Samuel Selvin, Fabio Di Troia, and Mark Stamp. Deep learning versus gist descriptors for image-based malware classification. In Paolo Mori, Steven Furnell, and Olivier Camp, editors, *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, ICISSP, pages 553–561, 2018.